

27.09.2022

Lecture 1

Tuesday

Algorithm Analysis = Complexity Analysis

There are basically 2 measures related to complexity analysis

- 1- Time complexity
 - 2- Space complexity
- more important

Problem - Algorithm

Distinction is important

⊗ Lower bound is defined for problems, not algorithms

<u>Problem</u>	<u>Algorithm</u>	
Sorting	Quick sort	
Lower bound comparison based: $\Omega(n \log n)$	Complexity: $O(n \log n)$	} So, there is no point to try to develop a better algorithm for sorting.
Sorting		

There are some problems whose lower bounds are very high:

Ex: Towers of Hanoi

↳ lower bound is $2^n \rightarrow$ This means that do not try to solve a computer to solve it

⊗ Feasible/infeasible, tractable/intractable, easy/hard, problems/algorithms

⊗ Knowing the execution time on a particular computer is not useful information. We also need to know the characteristics of that computer.

{ Algo. A executes in 10 seconds in type of computers
 Algo. A executes in 20 seconds in type of computers.

→ We cannot compare the algorithms

⊗ Algo A executes n complexity
 ⊗ Algo B executes $2n$ complexity

} independent from the computer

⊕ The type of the computer on which an algorithm operates is irrelevant. It does not change the nature of the algorithm.

⊗ Selecting an appropriate input parameter

Ex: Sorting, searching, finding maximum element in a list of integers

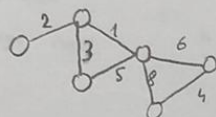
Input parameters: The number of elements in the list

Evaluating a polynomial: $p(n) = a_n \cdot x^n + \dots + a_1 x + a_0$

Multiplying 2 matrices: $\begin{bmatrix} & \\ & \end{bmatrix}_{n \times n} \begin{bmatrix} & \\ & \end{bmatrix}_{n \times n}$

Dijkstra

number of edges
number of vertices



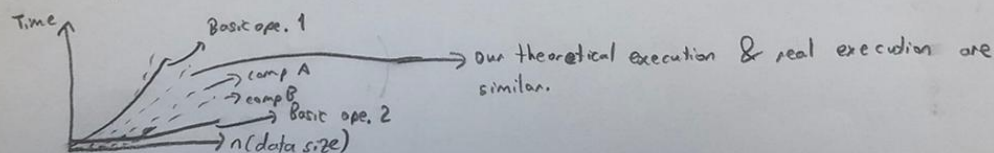
Graph Traversal Problem: Input = # of vertices
of edges

Basic Operation: Usually, the most important operation

The operation that characterizes the algorithm.

OR
an operation that the algorithm performs repeatedly
OR
the operation contributing the most to the total execution time

A basic operation is called realistic if the actual execution time on some computer is bounded by a linear function of the derived complexity



- Searching a list
 - Sorting a list
 - Finding max in a list
- } Basic operation: Comparison

- Multiplying 2 matrices

Basic operation: Multiplication → better
Addition can also be a basic operation
Because it takes much longer time than addition in the hardware

- Traversing a graph

Basic operation: Accessing/visiting a node

- Spell checking

Basic operation: # of dictionary look ups
↳ A higher level operation it characterizes better

For data size $n \Rightarrow$ We can analyse 3 types of complexity

- ① Best-case complexity
- ② Worst-case complexity
- ③ Average-case complexity

T_n : The set of all inputs of size n

$T(I)$: The # of basic operations performed by the algorithm for input I .

⊕ Best-case complexity of an algorithm: $B(n) = \min \{ T(I) ; I \in T_n \}$

⊕ Worst-case " " " " " : $W(n) = \max \{ T(I) ; I \in T_n \}$

⊕ Average-case " " " " " : $A(n) = \sum_{I \in T_n} T(I) P(I)$

$\underbrace{P(I)}_{\text{Probability of } I}$

=
Probability of occurrence
of input I

4

28.09.2022

Lecture 2

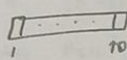
Wednesday

Average-case complexity

$$\begin{aligned}
 A(n) &= \sum_{I \in T_n} T(I) \cdot P(I) \\
 &= T(I_1) \cdot P(I_1) + T(I_2) \cdot P(I_2) + \dots \\
 &= \sum_{i=1}^{w(n)} i \cdot P_i
 \end{aligned}$$

$\downarrow$
 # of operations

$\rightarrow$ Probability that the algorithm performs i basic operations

Ex: An algorithm for searching in a list of size n $\rightarrow$ Suppose that $n=10$ 

10!

 $n=3$ 

3!

 $B(n)$ is very good $A(n)$ is very badCalculating $A(n)$: $\rightarrow$ We need to determine suitable values for the probabilities.

(i.e. a probability distribution for the input)

 $\rightarrow$ We need to know and apply elements of probability theory. $\rightarrow$ Formula for $A(n)$ usually involves summation or recurrence relation.

$$A(n) = \sum_{i=1}^n \left(\frac{i^n}{5} + \log i \right) \Rightarrow \text{we should convert the formula into "closed form"}$$

$$A(n) = \frac{n^3}{10} = \text{sth like this}$$

General Framework for the Analysis of an Algorithm

 $\rightarrow$ Decide on (a) parameter(s) indicating the input size. $\rightarrow$ Identify the algorithm's basic operation $\rightarrow$ Count how many times the basic operation is performed for an input of size n $\hookrightarrow$ In fact, complexity analysis means counting. $\rightarrow$ The most difficult step in this framework

For some algorithms, complexity does not depend on the nature of the data.

If this is the case, $A(n) = w(n) = B(n)$

If the result is in the form of a summation of a recurrence relation, then convert it to a closed-form expression by some mathematical manipulations.
(using approximations is also sometimes permissible)

Usually, iterative algorithms result in summations
 & recursive " " " recurrences

$A(n) = \dots \Rightarrow$ exact calculation

$A(n) \in \dots \Rightarrow$ approximate

⊗ Today, there exist some algorithms for which the complexities cannot be calculated

$$\left. \begin{aligned} A_1(n) &= 3n^2 + \frac{100 \log n}{\sqrt{n}} \\ A_2(n) &= \frac{10 + 2(n! - 5n)}{n^{3/2}} \end{aligned} \right\} \text{Which one is better?}$$

Principles

→ Look at the growth of the complexity function as $n \rightarrow \infty$

Analyzing the behaviour of an algorithm as the input size tends to infinity is called asymptotic analysis.

→ Look at the growth rate of the complexity function instead of the actual values it takes for different n 's (data sizes)

④ The rate of increase in the complexity function of an algorithm as the input size increases is called the growth rate.

Ex: $f_1(n) = \frac{n^2}{2} - \frac{n}{2}$
 $f_2(n) = n^2$ } compare their growth rates

n	$n/2$	$\frac{n^2}{2}$	$\frac{n^2}{2} - \frac{n}{2}$	n^2
10	5	50	45	100
1000	500	500000	499500	1000000
10000	5000	50000000		100000000
100000	50000	5000000000		10000000000

⊗ Eliminate low-order terms

④ Eliminate constant factor

$f_1(n)$ and $f_2(n)$ are

asymptotically equivalent

11

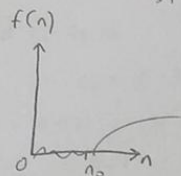
The complexity $f(n)$ of this algorithm is of order $g(n)$ $\Rightarrow$ we will say sth. like this

6

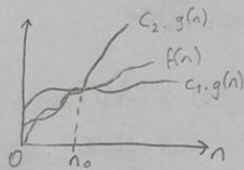
04.10.2022
Lecture 3
Tuesday

Let F be the set of all functions $f(n): \mathbb{N} \rightarrow \mathbb{R}$
that are eventually positive

there exists a number n_0
st $f(n) > 0 \quad \forall n > n_0$



- ⊛ Let $f(n)$ and $g(n) \in F$
 $f(n) \in \Theta(g(n))$ iff there exists positive constants $c_1 > c_2$ and n_0 such
that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ whenever $n \geq n_0$



⊛ $f(n)$ grows at the same rate as $g(n)$

$\Theta(g(n))$ represents a lot of functions

↳ Always a set, not a function itself
of
function

$$\Theta(n^2) = \{n^2, 5n^2, 6n^2, n^2 + \log n, \dots\}$$

$$\Theta(n^2) = \Theta(5n^2)$$

$$5n^2 \in \Theta(n^2) = \Theta(5n^2)$$

Ex: $n^2 \in \Theta(5n^2 + n)$ we need to prove that

$$c_1 \cdot (5n^2 + n) \leq n^2 \leq c_2 \cdot (5n^2 + n) \quad \text{for } n \geq n_0$$

If we can show that such c_1, c_2, n_0 exists, then it's the proof

$$c_1 = \frac{1}{10}, \quad c_2 = 1, \quad n_0 = 1$$

$$\frac{5n^2 + n}{10} \leq n^2 \leq 5n^2 + n \quad \forall n \geq 1$$

Ex: $n^2 \notin \Theta(n^3)$ Proof/ Suppose that $n^2 \in \Theta(n^3)$

Then $c_1 \cdot n^3 \leq n^2 \leq c_2 \cdot n^3 \quad \forall n \geq n_0$

$\exists c_1, c_2, n_0$

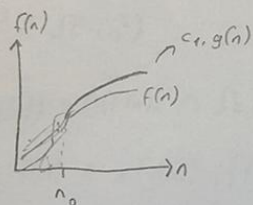
$$c_1 \cdot n \leq 1 \leq c_2 \cdot n \quad (\forall n \geq n_0)$$

0,001 $\downarrow$ X
1001

Θ determines an equivalence relation on the set of functions,

- reflexive : $f(n) \in \Theta(f(n))$
- symmetric : $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$
- transitivity : $f(n) \in \Theta(g(n))$ and $g(n) \in \Theta(h(n)) \Rightarrow f(n) \in \Theta(h(n))$

⊗ A function $f(n) \in O(g(n))$ iff there exists positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ whenever $n > n_0$



⊗ $f(n)$ grows at the same rate or slower than $g(n)$

O (big-Oh) notation gives us an upper bound for the complexity.

Ex: $5n^2 \in O(n^2)$

$5n^2 \leq c \cdot n^2 \Rightarrow$ Finding only one c and n_0 is satisfied,

$$[n_0 = 1, c = 10]$$

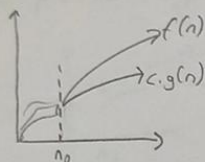
Ex: $n^2 \in O(n^3)$

For $n \geq 1, c = 2$

8

04.10.2022
Lecture 3
Tuesday

A function $f(n) \in \Omega(g(n))$ iff there exist positive constants c and n_0 such that $c \cdot g(n) \leq f(n)$ whenever $n \geq n_0$.



$f(n)$ grows at the same rate or faster than $g(n)$.

Ω notation gives us a lower bound for complexity.

* Matrix multiplication $\Rightarrow$

Classical way: $O(n^3)$

$$\begin{bmatrix} & \\ & \\ & \end{bmatrix}_{n \times n} \begin{bmatrix} & \\ & \\ & \end{bmatrix}_{n \times n} = \begin{bmatrix} & \\ & \\ & \end{bmatrix}_{n \times n}$$

linear $f(n)$

A better algorithm $\rightarrow O(n^{2.77})$

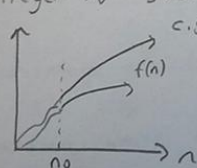
Problem complexity: $\Omega(n^2)$

Ex: $5n^2 \in \Omega(n^2)$

Ex: $n^3 \in \Omega(n^2)$

$$\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$$

A function $f(n) \in o(g(n))$ iff for every positive constant C , there is a positive integer n_0 such that $f(n) < C \cdot g(n)$ whenever $n > n_0$.



$f(n)$ grows slower than $g(n)$

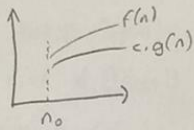
Ex: $n^2 \in o(n^3)$, $n^2 \in O(n^3)$, $n^2 \notin o(n^2)$

Ex: $n^2 \in O(5n^2)$
 $n^2 \notin o(5n^2)$

9

04.10.2022
lecture 3
Tuesday

⊗ A function $f(n) \in \omega(g(n))$ iff for every positive constant c , there is a positive integer n_0 such that $c \cdot g(n) < f(n)$ whenever $n \geq n_0$.



⊗ $f(n)$ grows faster than $g(n)$

Ex: $n^3 \in \omega(n^2)$
 $n^3 \notin \omega(5n^3)$

Asymptotic Notation - Relational Operators

$\Theta \leftrightarrow =$

$O \leftrightarrow \leq \Rightarrow$ The most frequently used

$\Omega \leftrightarrow \geq$

$o \leftrightarrow <$

$\omega \leftrightarrow >$

⊗ $f(n, m) \in O(g(n, m))$ iff there exists positive constants c and n_0 such that $f(n, m) \leq c \cdot g(n, m)$ whenever $n, m \geq n_0$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & ; \text{ if } f(n) \in o(g(n)) \\ c & ; \text{ where } c > 0 \text{ if } f(n) \in \Theta(g(n)) \\ \infty & ; \text{ if } f(n) \in \omega(g(n)) \\ 1 & ; \text{ if } f(n) \in \sim(g(n)) \end{cases}$$

tilde (strongly asymptotic to)

Disadvantage of limit approach

↓

Sometimes limits don't exist.

L'Hospital's Rule Let $f(x)$ and $g(x)$ be functions that are differentiable for sufficiently large real numbers x . If $\lim_{x \rightarrow \infty} f(x) = \infty$ and $\lim_{x \rightarrow \infty} g(x) = \infty$ OR if $\lim_{x \rightarrow \infty} f(x) = 0$ and $\lim_{x \rightarrow \infty} g(x) = 0$; then $\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{f'(x)}{g'(x)}$

Properties

→ set of functions

Let $f(n), g(n) \in \mathcal{F}$

- $f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n))$
- $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$
- $f(n) \in \Theta(g(n)) \Leftrightarrow f(n) \in O(g(n))$ and $f(n) \in \Omega(g(n))$
- $f(n) \in o(g(n)) \Leftrightarrow g(n) \in \omega(f(n))$
- $f(n) \in o(g(n)) \Rightarrow f(n) \in O(g(n))$
- $f(n) \in \omega(g(n)) \Rightarrow f(n) \in \Omega(g(n))$
- Given $c > 0$, $\Theta(f(n)) = \Theta(c \cdot f(n))$
- $f(n) \in O(g(n)) \Leftrightarrow O(f(n)) \subseteq O(g(n))$

Ex: $n \in O(n^2)$

subset

$$O(n) \subseteq O(n^2)$$

$$\rightarrow f(n) \in o(g(n)) \Leftrightarrow O(f(n)) \subset O(g(n))$$

proper subset

$$\rightarrow f(n) \in \sim(g(n)) \Rightarrow f(n) \in \Theta(g(n))$$

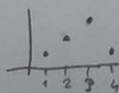
$$\rightarrow \Omega(f(n)) = O(g(n)) \Leftrightarrow \Theta(f(n)) = \Theta(g(n)) \Leftrightarrow \Omega(f(n)) = \Omega(g(n))$$

* Given two functions $f(n), g(n) \in \mathcal{F}$, $f(n)$ and $g(n)$ have the same order if $f(n) \in \Theta(g(n))$.

* $f(n)$ has smaller order than $g(n)$ if $O(f(n)) \subset O(g(n))$

* $f(n)$ and $g(n)$ are not comparable if $f(n) \notin O(g(n))$ and $g(n) \notin O(f(n))$

Ex: Functions involving
modulo operator
or
sinusoidal functions



$$O(n^2) = \{ \dots \}$$

They are infinite

Popularly Used Classes

→ fastest order

→ $O(1)$: constant order→ $O(\log \log n)$, $O((\log \log n)^{1.5})$, ...

↳ Poly-logarithmic order

→ $O(\log n) \Rightarrow$ Logarithmic order→ $\log_a n = \log_a b \cdot \log_b n$
for $a, b > 1$ → $O(n^a)$; where $0 < a < 1 \Rightarrow$ The famous one of this class is $O(\sqrt{n})$ n^a here grows faster than $\log n$
Consider $a = 0.01$

$$\lim_{n \rightarrow \infty} \frac{n^{0.01}}{\log n} = \frac{\infty}{\infty} \Rightarrow \text{use L'Hospital's Rule}$$

$$= \lim_{n \rightarrow \infty} \frac{0.01 \cdot n^{-0.99}}{\frac{1}{n}} = \lim_{n \rightarrow \infty} 0.01 \cdot n^{0.01} = \infty$$

 $\therefore n^{0.01}$ grows faster than $\log(n)$ → $O(n)$: linear order
used frequently

Ex: Scanning the entire list.

→ $O(n \cdot \log n)$ = Typical in
divide and
conquer algorithms
(Merge sort, quick sort)→ $O(n^2)$ = quadratic order $\Rightarrow$ Typical for algorithms
that have 2 nested loops

→ $O(n^a)$, where $0 < a < 1$

→ $O(n)$, linear order ⇒ used frequently

→ $O(n \log n)$, typical in divide-and-conquer algorithms

→ $O(n^2)$, quadratic order

→ $O(n^3)$, cubic order

→ $O(a^n)$ for $a > 1$ exponential order

a^n grows faster than n^a

$a^{\sqrt{n}}$, $a^{\log^2 n}$ are also exponential. The power need not be n .

2^n ⇒ Typical for algorithms that generate all possible subsets of a given set.

Exponential algorithms are very slow.

$O(n \cdot a^n)$; $a > 1$

→ $O(n!)$ Factorial order
very very slow algorithms

→ $O(a^{n^b})$ for $a, b > 1$

$O(1) \subset O(\log \log n) \subset O(\log n) \subset O(n^a) \subset O(n) \subset O(n \log n) \subset O(n^a)$
for $0 < a < 1$ Polynomial $a > 1$

$O(a^{n^b}) \supset O(n!) \supset O(n \cdot a^n) \supset O(a^n)$

⊗ An algorithm whose complexity is at most $O(f(n))$, where $f(n)$ is a polynomial of n is called a polynomial time algorithm.

⊗ An algorithm whose complexity is at least $\Omega(a^n)$; where $a > 1$, is called an exponential time algorithm.

⊗ This distinction is the most important thing in the running time of algorithms.

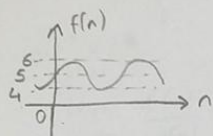
⊗ Polynomial time algorithms are feasible algorithms, whereas exponential algorithms are infeasible.

1.13

11.10.2022

Lecture 5
Tuesday

Ex: $f(n) = 5 + \sin(n)$



It is always bounded above by 6
It " " " " below by 4

$$\begin{aligned} f(n) &\leq 6 \\ f(n) &\geq 4 \end{aligned} \Rightarrow f(n) \in O(1)$$

$$\begin{aligned} f(n) &\leq c \cdot 1 & \text{for } n \geq n_0 \\ &\downarrow & \downarrow \\ &6 & 1 \end{aligned}$$

$\therefore f(n) \in \Theta(1)$

$$f(n) \in \Omega(1) \text{ because } c \cdot 1 \leq f(n) \text{ for } n \geq n_0$$

Ex: Prove that $\sum_{k=1}^n k^p$ is $\Theta(n^{p+1})$ for $p > 1$ a fixed constant.

→ First find an upper bound

$$\sum_{k=1}^n k^p = 1^p + 2^p + 3^p + \dots + n^p \leq n \cdot n^p = n^{p+1}$$

n terms

Each term is at most n^p

$1^p < n^p$
 $2^p < n^p$

$$\sum_{k=1}^n k^p \in O(n^{p+1})$$

→ Find the lower bound

$$\sum_{k=1}^n k^p$$

$$\lfloor n \rfloor : \text{Floor} \quad \lfloor 7.2 \rfloor = 7$$

$$\lceil n \rceil : \text{Ceiling} \quad \lceil 7.2 \rceil = 8$$

$$\sum_{k=1}^n k^p = \underbrace{\sum_{k=1}^{\lfloor \frac{n}{2} \rfloor} k^p}_A + \underbrace{\sum_{k=\lfloor \frac{n}{2} \rfloor + 1}^n k^p}_B > \underbrace{\sum_{k=\lfloor \frac{n}{2} \rfloor + 1}^n k^p}_B$$

$$(\lfloor \frac{n}{2} \rfloor + 1)^p + (\lfloor \frac{n}{2} \rfloor + 2)^p + \dots + n^p$$

There are $n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil$ terms

$$\therefore \sum_{k=1}^n k^p \in \Omega(n^{p+1})$$

$$\text{Each term } > (\lfloor \frac{n}{2} \rfloor + 1)^p > (\frac{n}{2})^p$$

$$\text{The total sum } > \frac{n}{2} \cdot \left(\frac{n}{2}\right)^p = \frac{n^{p+1}}{2^{p+1}}$$

Ex: $f(n) = \frac{n(n+1)}{2}$ find the asymptotic relationship between these 2 functions
 $g(n) = n^2$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{n(n+1)}{2n^2} = \frac{1}{2} \Rightarrow f(n) \in \Theta(g(n))$$

Ex: Show that any polynomial function has smaller order than any exponential function, i.e. $n^k \in o(a^n)$, or, in other words, $O(n^k) \subset O(a^n)$ for $k > 0, a > 1$

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{n^k}{a^n} &\stackrel{\text{L'Hopital}}{=} \lim_{n \rightarrow \infty} \frac{k \cdot n^{k-1}}{(\ln a) \cdot a^n} \stackrel{\text{L'Hopital}}{=} \lim_{n \rightarrow \infty} \frac{k \cdot (k-1) \cdot n^{k-2}}{(\ln a)^2 \cdot a^n} \\ &\vdots \\ &\stackrel{\text{Apply L'Hopital consecutively}}{=} \lim_{n \rightarrow \infty} \frac{k \cdot (k-1) \cdot \dots \cdot 1}{(\ln a)^k \cdot a^n} \\ &= \lim_{n \rightarrow \infty} \frac{k!}{(\ln a)^k \cdot a^n} = 0 \end{aligned}$$

Ex: Compare orders of growth of $n!$ and 2^n .

$$\lim_{n \rightarrow \infty} \frac{n!}{2^n} = \lim_{n \rightarrow \infty} \sqrt{2\pi n} \cdot \left(\frac{n}{2e}\right)^n = \infty$$

Stirling's Formula

$$n! \approx \sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n$$

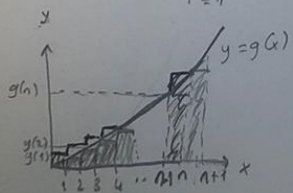
$$\therefore n! \in \omega(2^n)$$

for large n

* In analyzing an algorithm, we usually arrive at a summation Σ and then convert this summation into closed form.

* Sometimes this Σ is too complex. At that time, we can squeeze the summation between 2 integrals.

Let $f(n) = \sum_{i=1}^n g(i)$ where g is a nondecreasing function.



$$g(1) + g(2) + \dots + g(n) = f(n)$$

$$\int_1^{n+1} g(x) dx \Rightarrow \text{The entire area under the curve}$$

$$g(1) + g(2) + \dots + g(n) = \sum_{i=1}^n g(i) = f(n)$$

$$f(n) \geq \int_1^n g(x) dx$$

$$f(n) \leq \int_1^{n+1} g(x) dx$$

$$\int_1^n g(x) dx \leq \sum_{i=1}^n g(i) \leq \int_1^{n+1} g(x) dx$$

This is valid only if g is a nondecreasing function

15

Ex

$$f(n) = \sum_{i=1}^n i^2$$

Non decreasing ✓

$$\int_0^n g(x) dx \leq f(n) \leq \int_1^{n+1} g(x) dx$$

$$\int_0^n x^2 dx \leq f(n) \leq \int_1^{n+1} x^2 dx$$

$$\frac{x^3}{3} \Big|_0^n \leq f(n) \leq \frac{x^3}{3} \Big|_1^{n+1}$$

$$\frac{n^3}{3} \leq f(n) \leq \frac{(n^3+1)^3-1}{3}$$

$$f(n) \in \Omega(n^3)$$

$$f(n) \in O(n^3)$$

$$f(n) \in \Theta(n^3)$$

11.10.2022

Lecture 5

Tuesday

Ex: Sum of logarithms

Method 1

$$L(n) = \sum_{i=1}^n \log(i) = \log(n!)$$

$$L(n) = \underbrace{\log 1 + \log 2 + \dots + \log n}_{n \text{ terms}} \leq n \cdot \log n \in O(n \log n)$$

For lower bound:

$$\sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \log(i) + \sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^n \log(i) \geq \sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^n \log(i)$$

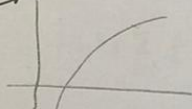
$$= \log(\lfloor \frac{n}{2} \rfloor + 1) + \log(\lfloor \frac{n}{2} \rfloor + 2) + \dots + \log(n)$$

$$n - \lfloor \frac{n}{2} \rfloor = \lceil \frac{n}{2} \rceil \geq \frac{n}{2} \text{ terms}$$

$$\text{Each term } \geq \log\left(\frac{n}{2}\right) \dots \in \Omega(n \log n)$$

Method 2
integrating $\log(i)$ is a non-decreasing function

$$\therefore L(n) \in \Theta(n \log n)$$



$$\int_0^n \ln(x) dx \leq L(n) \leq \int_1^{n+1} \ln(x) dx$$

$$(x \cdot \ln x)' - x' = \ln x$$

$$(x \cdot \ln x - x) \Big|_0^n \leq L(n) \leq (x \cdot \ln x - x) \Big|_1^{n+1}$$

$$n \cdot \ln(n) - n \leq L(n) \leq (n+1) \cdot \ln(n+1) - (n+1) - (0-1)$$

$$(n+1) \cdot \ln(n+1) - n$$

$$L(n) \in \Theta(n \log n)$$

Stirling's Formula

$$e^{n \cdot \ln(n) - n} \leq e^{\ln(n!)} \leq e^{(n+1) \cdot \ln(n+1) - n}$$

$$\frac{e^{n \cdot \ln(n)}}{e^n} \leq n! \leq \frac{e^{(n+1) \cdot \ln(n+1)}}{e^n}$$

Note!

$$n \cdot \log_a(n) = n^{\frac{1}{a}}$$

17

191

12.10.2022

Lecture 6
Wednesday

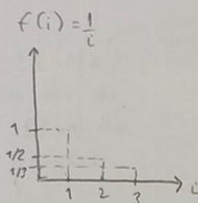
Harmonic Series

$$H(n) = \sum_{i=1}^n \frac{1}{i}$$

$$H(1) = 1$$

$$H(2) = 1 + \frac{1}{2} = 1.5$$

$$H(3) = 1 + \frac{1}{2} + \frac{1}{3} = 1.833...$$



$\times$ not non-decreasing
in fact non-increasing

$$\int_1^{n+1} g(x) dx \leq H(n) \leq \int_0^n g(x) dx \quad \text{for a non-increasing function}$$

$$\int \frac{1}{x} = \ln x$$

$$\ln(x) \Big|_1^{n+1} \leq H(n) \leq \ln(x) \Big|_0^n$$

$$\ln(n+1) - \underbrace{\ln 1}_0 \leq H(n) \leq \underbrace{\ln n - \ln 0}_{-\infty}$$

So this method here works for finding a LB, but not for an upper bound

↓
So we should find another way to find an upper bound

$$H(n) = \sum_{i=1}^n \frac{1}{i}$$

$$= 1 + \sum_{i=2}^n \frac{1}{i}$$

$$H(n) \leq 1 + \int_1^n g(x) dx$$

$$\leq 1 + \ln(x) \Big|_1^n$$

$$H(n) \leq 1 + \ln(n)$$

$$\leq 1 + \ln(n) - \underbrace{\ln 1}_0$$

$$\ln(n+1) \leq H(n) \leq \ln(n) + 1$$

$$\ln(n) \leq \ln(n+1) \leq 1 + \ln(n) \leq \ln(n) + 1 \leq 2 \ln(n) \quad \therefore H(n) \in \Theta(\log n)$$

12.10.2022
lecture 6
Wednesday

Linear (Sequential) Search

Pseudocode 1:

```
function LinearSearch (L[1:n], x) ↗ searched  
element  
  for i = 1 to n do  
    if (L[i] = x) then  
      return (i)  
    end if  
  end for  
  return ∅  
end
```

A comparison-based searching or sorting algorithm is an algorithm that is based on making comparisons involving list elements and then making decisions based on these comparisons.

Best case: If $x = L[1]$, then best case occurs.

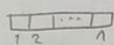
$$B(n) = 1 \in O(1)$$

$$\Theta(1)$$

Worst case: If $x = L[n]$ or x does not exist in the list, then worst case occurs.

$$W(n) = n \in \Theta(n)$$

Average case: Assume that the prob. of a successful search is p , where $0 \leq p \leq 1$



uniform
prob.
distr.

A classical assumption $\rightarrow$ Assume that x can equally likely be found in any position.
Assume that the elements in the list are distinct.

Prob. that i operations will be done, i.e. prob. that x occurs in the i th position

$$A(n) = \sum_{i=1}^{W(n)} i \cdot p_i = \left\{ \sum_{i=1}^{n-1} i \cdot \frac{p}{n} \right\} + \left\{ n \cdot \left(\frac{p}{n} + 1 - p \right) \right\}$$

= Prob. that $x = L[i] = \frac{p}{n}$ for $1 \leq i \leq n-1$

$$= \frac{p}{n} \cdot \sum_{i=1}^{n-1} i + p + n(1-p)$$

Prob. that $x = L[i]$ or
prob. that x is not in the list

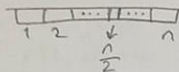
$$= \frac{p}{n} + (1-p) \text{ for } i=n$$

$$\rightarrow \frac{n(n-1)}{2} \cdot \frac{p}{n} + p + n - np$$

$$= \left(\frac{1-p}{2} \right) n + \frac{p}{2} \in \Theta(n) \rightarrow \text{because } p \text{ is a constant}$$

Binary search

BS assumes that the list is sorted. Binary search does not work,



Best case: $B(n) = 1 \in \Theta(1) \Rightarrow$ if x is equal to the middle element

Worst case: It is not easy to analyze the worst case for the general n .

So, firstly, suppose that $n = 2^k - 1$, where k is an integer. If x is not in the list or $x = L[1]$ or $x = L[n]$, then worst case occurs.

Compare x with $L[2^{k-1}]$, then $L[2^{k-2}], \dots, L[1]$

There are k comparisons $\Rightarrow$ Worst-case complexity

$$n = 2^k - 1 \Rightarrow 2^k = n + 1$$

$$k = \log_2(n+1) \Rightarrow w(n) = \log_2(n+1) \text{ for } n = 2^k - 1$$

Now, let's generalize this result as follows, so $w(n) = \dots$ for all n

$$2^{i-1} - 1 \leq n \leq 2^i - 1$$

* $w(n)$ is a non-decreasing function.

$$w(2^{i-1} - 1) \leq w(n) \leq w(2^i - 1)$$

$$\log_2(2^{i-1} - 1 + 1) \leq w(n) \leq \log_2(2^i - 1 + 1)$$

$$i-1 \leq w(n) \leq i$$

$$2^{i-1} \leq n+1 \leq 2^i$$

$$i-1 \leq \log_2(n+1) \leq i$$

$$\log_2(n+1) - 1 \leq i-1 \leq w(n) \leq i \leq \log_2(n+1) + 1$$

$$\log_2(n+1) - 1 \leq w(n) \leq \log_2(n+1) + 1$$

There are $w(n) \in \Theta(\log n)$ for all n

④ In fact, this method is called interpolate

21

18.10.2022

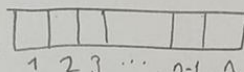
Lecture 7

Tuesday

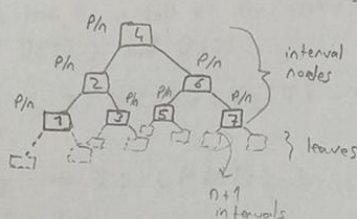
Average case

- Assume that the prob. of a successful search is p ($0 \leq p \leq 1$)
 → " " it is equally likely that x can be found in any position.
 → If x is not in the list, it is equally likely that it can occur in any one of the following $n+1$ intervals:

$$x < L[1], L[1] < x < L[2], \dots, L[n-1] < x < L[n]$$



7 element list

This tree has n interval nodes and $n+1$ leaf nodescorresponding the $n+1$ unsuccessful situationsProb. that x can be found in any position $L[i] : \frac{p}{n}$; for $1 \leq i \leq n$

Average number of comparisons:

$$\text{if } x = L[4] \Rightarrow \frac{p}{n} \cdot 1 \text{ comparisons}$$

$$\text{if } x = L[2] \Rightarrow \frac{p}{n} \cdot 2 \text{ comparisons}$$

$$= \frac{p}{n} \cdot 1 + \frac{p}{n} \cdot 2 + \dots + \frac{p}{n} \cdot 4 = \frac{p}{n} \cdot (1+2+2+3+3+3+4)$$

In general

This corresponds to the internal path length of the tree

Internal Path Length: The sum of the lengths of the paths from the root to the internal nodes.

$$\text{Average number of comparisons} = \frac{p}{n} (\text{IPL}(T) + n)$$

The prob (when x is not in the list) x falls into any one of the $n+1$ intervals = $\frac{1-p}{n+1}$

$$\text{Average number of comparisons} : \left(\frac{1-p}{n+1}\right) \cdot 3 + \left(\frac{1-p}{n+1}\right) \cdot 3 + \dots = \left(\frac{1-p}{n+1}\right) \cdot (3+3+\dots)$$

Leaf Path Length: The sum of the lengths of the paths from the root to the leaf nodes.

$$\text{Average \# of comparisons for the unsuccessful case} = \left(\frac{1-p}{n+1}\right) \cdot \text{LPL}(T)$$

22

18.10.2022

Lecture 7
Tuesday

$$A(n) = \sum_{I \in T_n} T(I) \cdot P(I) = \frac{p}{n} (IPL(T) + n) + \left(\frac{1-p}{n+1} \right) (LPL(T))$$

Definition: A 2-tree is a binary tree such that every node that is not a leaf has exactly 2 children



Definition: A binary tree T is called a full binary at level i if T contains the maximum number 2^i nodes at that level; $0 \leq i \leq D(T)$



Depth

19.10.2022

Lecture 8
Wednesday

Definition: A 2-tree T is full at the second-deepest level if all leaf nodes are in the levels $D(T)-1$ and $D(T)$

* For a 2-tree T , $IPL(T) = LPL(T) - 2I$; where I is the number of internal nodes.

For any binary tree T ; $LPL(T) \geq L \cdot \lfloor \log_2 L \rfloor + 2(L - 2^{\lfloor \log_2 L \rfloor})$ where L is the number of leaf nodes.

→ This becomes an equality for a 2-tree full at the second deepest level

$$A(n) = \frac{p}{n} (LPL(T) + n) + \left(\frac{1-p}{n+1} \right) LPL(T)$$

$$= \left(\frac{p}{n} + \frac{1-p}{n+1} \right) \cdot LPL(T) - p$$

$$\geq \left(\frac{p}{n} + \frac{1-p}{n+1} \right) \cdot \left[(n+1) \cdot \lfloor \log_2 (n+1) \rfloor + 2 \cdot \left((n+1) - 2^{\lfloor \log_2 (n+1) \rfloor} \right) \right] - p$$

$$> \left(\frac{p}{n} + \frac{1-p}{n+1} \right) \cdot (n+1) \cdot \lfloor \log_2 (n+1) \rfloor - p$$

$$\sim \frac{1}{2}$$

$$\sim \log n$$

$$\in \Omega(\log n)$$

$$A(n) \leq w(n) \in O(\log n)$$

$$A(n) \in O(\log n)$$

$$A(n) \in \Omega(\log n)$$

$$A(n) \in \Theta(\log n)$$

75.

23

1970, 2022

Lecture 8
Wednesday

Interpolation

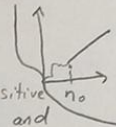
We know that $f(n) \in \Theta(g(n))$ for $n = b^k$ ($k \geq 0$; $b \geq 1$)

$$[f(b^n) \in \Theta(g(b^n)) \text{ for } n \in \mathbb{N} \text{ (for all } n)]$$

We want to show that $f(n) \in \Theta(g(n))$ for all $n \in \mathbb{N}$.

⊗ A function $h(n)$ is called eventually non-decreasing if there exists an integer n_0 such that $h(n) \leq h(m)$ whenever $m \geq n$, $n \geq n_0$.

⊗ For a function $h(n)$, if $h(c \cdot n) \in \Theta(h(n))$ for any positive constant c , then $h(n)$ is called Θ -invariant under scaling and $\Theta(h(n))$ is called a scale invariant class.



Ex: $h(n) = n^2 \in \Theta(n^2)$

$$h(c \cdot n) = \underbrace{c^2}_{\text{constant}} \cdot n^2 \in \Theta(n^2)$$

Ex: $h(n) = \log n \in \Theta(\log n)$

$$\begin{aligned} h(c \cdot n) &= \log(c \cdot n) \\ &= \underbrace{\log c}_{\text{constant}} + \log n \end{aligned}$$

751
224

26.10.2022
Wednesday
Lecture 9

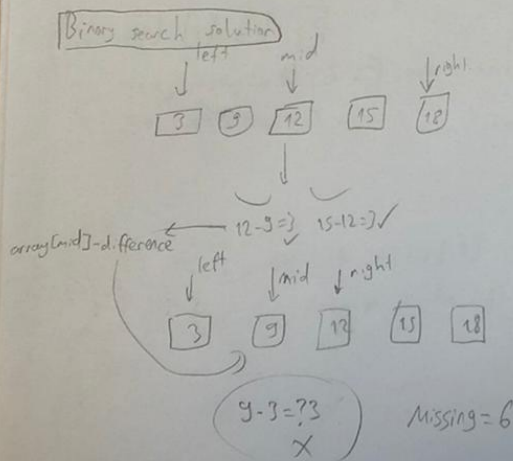
Problem 1

Construct an algorithm to find the missing element in a regularly increasing array.

Example:

2 4 6 8 12 $\rightarrow$ missing element is 10
3 9 12 15 18 $\rightarrow$ missing element is 6

differences []
n $\rightarrow$ length of array
for i from 0 to n-1:
 append (array[i+1] - array[i]) to differences
difference $\leftarrow$ mostly seen element in differences
for i from 0 to n-1:
 if (array[i+1] - array[i]) != difference:
 return array[i] + difference



difference = 3

a = right - mid = 4 - 2 = 2 (Number of additions)
array[mid] + a * difference = ? array[right]
12 + 2 * 3 = 18 ✓
array[mid] - a * difference = ? array[left]
3 = ? 12 - 2 * 3 False

Best case: $\Omega(1)$
Worst case: $O(\log n)$
Average case: $\Theta(\log n)$

difference $\leftarrow$ (last - first) / (length of array)
for i from 0 to n-1:
 if (array[i+1] - array[i]) != difference:
 return array[i] + difference
Best case: $\Omega(1)$
Worst case: $O(n)$
Average case: $\Theta(n)$

Ex: Construct an algorithm to find the maximum difference between two elements such that the smaller element appears before the larger one.

6 10 1 7 5 $\rightarrow$ max difference is $7-1=6$
 15 6 2 3 5 $\rightarrow$ max difference is $5-2=3$

A solution with $O(n^2)$ complexity:

```
findMaxDiff(array)
  difference  $\leftarrow 0$ 
  n  $\leftarrow$  length of array
  for i from 0 to n-1:
    for j from i+1 to n
      if array[j]
```

A solution with $O(n)$ complexity:

6 10 1 7 5

max = array[1] = 5 $\Rightarrow$ max = current = 7

max_difference = 0 $\Rightarrow$ max(max_diff, max_element - current_element) = max_diff = 0, 7-1 = 6

```
max_diff  $\leftarrow 0$ 
max_ele  $\leftarrow$  array[1]
for i from n-2 to 0
```

Ex: $h(n) = 2^n \in \Theta(2^n)$

$$h(c \cdot n) = 2^{c \cdot n}$$

$$= (2^c)^n \notin \Theta(2^n)$$

$$\in \Theta(2^{cn})$$

⊗ Exponential functions are not Θ -invariant

⊗ Suppose that $f(b^n) \in \Theta(g(b^n))$ for $b > 1$ and $n \in \mathbb{N}$. Suppose that $f(n)$ and $g(n)$ are eventually non-decreasing and $g(n)$ is Θ -invariant under scaling. Then $f(n) \in \Theta(g(n)) \quad \forall n \in \mathbb{N}$.

Ex: $w(n) \in \Theta(\log n)$ for $n = 2^k - 1$

Recall $\log_2(n+1) - 1 < w(n) < \log_2(n+1) + 1$

$w(n)$ is non-decreasing
 $\log(n)$ is also "
 $\log(n)$ is Θ -invariant
 $w(n) \in \Theta(\log n) \quad \forall n \in \mathbb{N}$

INSERTION SORT

procedure InsertionSort($L[1:n]$)

for $i = 2$ to n do

current = $L[i]$

position = $i - 1$

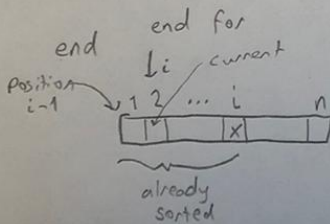
while (position > 1) and (current $< L[\text{position}]$) do

$L[\text{position} + 1] = L[\text{position}]$

position = position - 1

end while

$L[\text{position} + 1] = \text{current}$



Best case: The basic operation executes once and the while loop terminates. This occurs if the input is already sorted.

$$B(n) = \sum_{i=2}^n 1 = n - 1 \in \Theta(n)$$

for loop always executes

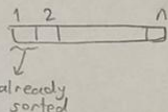
Worst case: For each iteration of the for loop, the basic operation is executed maximum number of times. This situation occurs if the list $L[1:n]$ is already sorted in reverse order.

$$w(n) = \sum_{i=2}^n (i-1) = \sum_{i=1}^{n-1} i = \frac{n \cdot (n-1)}{2}$$

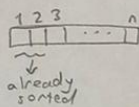
Average case

Step 1:

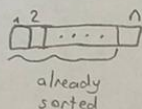
- of the for loop:

Place $L[2]$ into its proper position

Step 2:

Place $L[3]$ " " " "

⋮

Step $(n-1)$:Place $L[n]$ into its proper positionLet $T_i = \#$ of basic operations at step i .for $1 \leq i \leq n-1$

$$T = T_1 + T_2 + \dots + T_{n-1} = \sum_{i=1}^{n-1} T_i$$

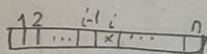
$$A(n) = E[T] = E\left[\sum_{i=1}^{n-1} T_i\right]$$

$$= \sum_{i=1}^{n-1} E[T_i] = E[T_1] + E[T_2] + \dots + E[T_{n-1}]$$

Calculate $E[T_i]$:

$$E[T_i] = \sum_{j=1}^i j \cdot P(T_i = j)$$

Random variable

Prob. that there are j comparisons in the i th step1 comparison will occur if $x = L[i] > L[i-1]$ 2 comparisons " " if $L[i-2] < x < L[i-1]$ 3 " " " if $L[i-3] < x < L[i-2]$ ⋮ " " " if $L[1] < x < L[2]$ i " " " if $x < L[1]$

⊛ We assume that each of these intervals is equally likely.

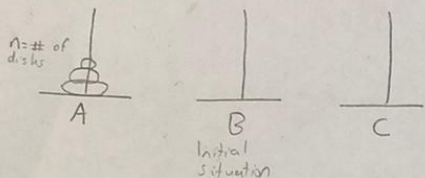
⊛ There are $i+1$ possible cases

$$P(T_i = j) \approx \begin{cases} \frac{1}{i+1}, & \text{if } 1 \leq j \leq i-1 \\ \frac{2}{i+1}, & \text{if } j = i \end{cases}$$

Stable final position: The unique final position of the element after any stable sorting algorithm is applied

Then insertion sort performs at most $2k(n-1)$ comparisons when sorting the list.

Towers of Hanoi:

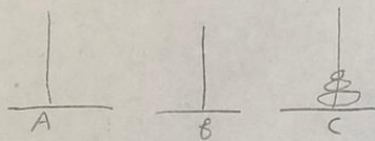


Rule 1: At each step, you can move only the pack on top of the other packs

We can move only 1 pack at a time

Rule 2: We cannot place a larger disk on top of a smaller one

Final Situation: Goal of algorithm



procedure Towers(A, B, C, n)

if $n = 1$ then
write ("Move from A to C")

else

call Towers(A, C, B, $n-1$) // Move $n-1$ disks from A to B
write ("Move from A to C") // Move the largest disk from A to C
call Towers(B, A, C, $n-1$) // Move $n-1$ disks from B to C

end

30

08.11.2022

Tuesday

Lecture 12

$T(n)$ = Complexity of Towers of Hanoi for an input of size n

$$T(n) = 2 \cdot T(n-1) + 1 \quad \text{for } n > 1$$

$$T(1) = 1$$

$$T(2) = 2 \cdot T(1) + 1 = 2 \cdot 1 + 1 = 3$$

$$T(3) = 2 \cdot T(2) + 1 = 2 \cdot (2 \cdot T(1) + 1) + 1 = 2^2 \cdot T(1) + 2 \cdot 1 + 1$$

$$T(4) = 2 \cdot T(3) + 1 = 2^3 \cdot T(1) + 2^2 + 2 + 1$$

$$T(n) = 2^{n-1} \cdot T(1) + 2^{n-2} + 2^{n-3} + \dots + 2^1 + 2^0$$

$$T(n) = \sum_{k=0}^{n-1} 2^k$$

$$= \frac{2^n - 1}{2 - 1} = 2^n - 1 \in \Theta(2^n)$$

$$\sum_{k=0}^{n-1} a^k = a^0 + a^1 + \dots + a^{n-1} = \frac{a^n - 1}{a - 1}$$

$$\sum_{i=0}^{n-1} x^i = \frac{x^n - 1}{x - 1} \quad \text{for } x \neq 1, n > 0, x \in \mathbb{R}^+$$

General Form of a Recurrence Relation

$$x(n) = c_1 \cdot x(n-1) + c_2 \cdot x(n-2) + \dots + c_k \cdot x(n-k) + f(n)$$

$x_0, \dots, x_{k-1}$ are given
initial conditions

$$x(n) = c \cdot x\left(\frac{n}{b}\right) + f(n); \quad \text{where } x(1) \text{ is given}$$

k^{th} order recurrence relations
 k is usually 1 or 2

$c_1, c_2, \dots \Rightarrow$ denotes the # of recursive calls

$f(n)$'s denote the extra operations in the algorithms

Forward Substitution

Ex: $x(n) = 2 \cdot x(n-1) + 1$; $n > 1$, $x(1) = 1$

$$\begin{aligned} x(1) &= 1 \\ x(2) &= 3 \\ x(3) &= 7 \\ x(4) &= 15 \\ x(5) &= 31 \\ &\vdots \end{aligned}$$

We try to observe a pattern

Guess $x(n) = 2^n \cdot x$

Guess $x(n) = 2^n - 1$

$$x(n-1) = 2^{n-1} - 1$$

$$x(n) = 2 \cdot x(n-1) + 1$$

$$2^n - 1 = 2 \cdot (2^{n-1} - 1) + 1$$

$$= 2^{n-2} + 1 = 2^n - 1 \quad \forall n$$

Backward Substitution

Ex: $x(n+1) = b_{n+1} \cdot x(n) + c_{n+1}$; $n \geq 0$, $x(0)$ is given

$$\begin{aligned} &\vdots \\ x(n+1) &= b_{n+1} \dots b_1 \cdot x_0 + b_{n+1} \dots b_2 \cdot c_1 + b_{n+1} \dots b_3 \cdot c_2 + \dots + c_{n+1} \end{aligned}$$

We can apply a trick to the original formula to obtain another formula

→ Make a change of variable that leads to a new recurrence

$$x(n) = b_1 b_2 \dots b_n \cdot y(n) ; n \geq 1 ; y(0) = x(0)$$

$$b_1 \dots b_{n+1} \cdot y(n+1) = b_{n+1} b_1 \dots b_n \cdot y(n) + c_{n+1}$$

We make the coefficients the same, so that we can eliminate them.

Now, this can easily be solved by backward substitution.

$$y(n+1) = y(n) + d_{n+1} \quad \text{where } d_{n+1} = \frac{c_{n+1}}{b_1 \dots b_{n+1}}$$

$y(0)$ is given

→ Solve the new recurrence

$$y(n) = y(0) + \sum_{i=1}^n d_i$$

→ Return to the original variable

$$x(n) = b_1 b_2 \dots b_n \left[y(0) + \sum_{i=1}^n d_i \right]$$

E x: $x(n+1) = 3 \cdot x(n) + 12$; $n \geq 0$; $x(0) = 0$

$$x(n) = 3^n \cdot y(n)$$

⊗ $3^{n+1} \cdot y(n+1) = 3^{n+1} \cdot y(n) + 12$; $y(0) = 0$; $n \geq 0$

$$y(n+1) = y(n) + \frac{n}{3^{n+1}} \Rightarrow y(n) = \sum_{i=1}^{n-1} \frac{i}{3^{i+1}}$$

$$\Rightarrow x(n) = 3^n \cdot \sum_{i=1}^{n-1} \frac{i}{3^{i+1}}$$

if we cannot solve the summation exactly, we can squeeze it between two integrals and find the Θ solution.

⊗ $x(n+1) = a \cdot x(n) + b \cdot x(n-1)$; $n \geq 1$, $x(0)$ & $x(1)$ are given

2nd order recurrence relation

$\alpha^2 = a \cdot \alpha + b \Rightarrow$ Characteristic equation of this recurrence.

↓
Solve this equation to find the roots for α and then we express the solution of the recurrence in terms of these roots

$$\alpha^{n+1} = a \cdot \alpha^n + b \cdot \alpha^{n-1}$$

↓

Divide all by α^{n-1}

$\alpha^2 = a \cdot \alpha + b$ Roots: α^+ and α^-

i.) if both α^+ and α^- are real and distinct, then

$$x(n) = c_1 \cdot \alpha_+^n + c_2 \cdot \alpha_-^n \text{ for some } c_1, c_2$$

↓
we can find these constants from the initial conditions

33

08.11.2022

Tuesday
Lecture 13ii-) if $\alpha_+ = \alpha_-$ (say, α) and it is real, then

$$x(n) = c_1 \cdot \alpha^n + c_2 \cdot n \cdot \alpha^n$$

iii-) If the roots are complex

$$\Downarrow$$

it won't occur in our algorithm analysis

Ex: Recurrence for Fibonacci Number

$$f(n+1) = f(n) + f(n-1) \quad n > 1, f(0) = 0$$

(characteristic equation

$$\therefore \alpha^2 = \alpha + 1$$

$$\alpha^2 - \alpha - 1 = 0$$

$$\text{roots} = \alpha_+ = \frac{1+\sqrt{5}}{2}, \quad \alpha_- = \frac{1-\sqrt{5}}{2}$$

$$f(n) = c_1 \cdot \alpha_+^n + c_2 \cdot \alpha_-^n = c_1 \cdot \left(\frac{1+\sqrt{5}}{2}\right)^n + c_2 \cdot \left(\frac{1-\sqrt{5}}{2}\right)^n$$

$$f(0) = 0 \Rightarrow c_1 + c_2 = 0$$

$$f(1) = 1 \Rightarrow c_1 \cdot \left(\frac{1+\sqrt{5}}{2}\right) + c_2 \cdot \left(\frac{1-\sqrt{5}}{2}\right) = 1$$

$$c_1 + c_1 \cdot \sqrt{5} + c_2 - c_2 \cdot \sqrt{5} = 2$$

$$c_1 + c_2 + c_1 \cdot \sqrt{5} - c_2 \cdot \sqrt{5} = 2$$

$$c_1 - c_2 = \frac{2\sqrt{5}}{5}$$

$$+ \quad c_1 + c_2 = 0$$

$$c_1 = \frac{\sqrt{5}}{5} = \frac{1}{\sqrt{5}}$$

$$c_2 = \frac{-\sqrt{5}}{5} = -\frac{1}{\sqrt{5}}$$

$$\rightarrow f(n) = \frac{\sqrt{5}}{5} \cdot \left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}} \cdot \left(\frac{1-\sqrt{5}}{2}\right)^n$$

34

08.11.2022
Tuesday
Lecture 12

$$\text{Ex: } x(n+1) = 2 \cdot x(n) - x(n-1) ; n \geq 1$$

$$x(0) = 1$$

$$x(1) = 5$$

$$\alpha^2 = 2\alpha - 1$$

$$\alpha^2 - 2\alpha + 1 = 0$$

$$(\alpha - 1)^2 = 0$$

$$\Downarrow$$

$$\alpha = \alpha^+ = \alpha^- = 1$$

$$x(n) = c_1 \cdot \alpha^n + c_2 \cdot n \cdot \alpha^n$$

$$x(n) = c_1 + c_2 \cdot n$$

$$x(0) = \boxed{c_1 = 1}$$

$$x(1) = \boxed{c_1 + c_2 = 5}$$

$$\boxed{c_2 = 4}$$

⊗

$$\boxed{x(n) = 4n + 1}$$

Ex:

$$\textcircled{1} \text{ One type of recurrence } \Rightarrow x(n) = a \cdot x(n-1) + f(n) ; x(1) = b$$

$$\Downarrow$$

Decrease-by-1 algorithms (like insertion sort)

$$x(n) = a \cdot x\left(\frac{n}{b}\right) + f(n) ; \text{ where } x(1) = c ; b > 1$$

$$b = 2 \text{ usually}$$

$$\text{if } a = 1 \Rightarrow \text{decrease-by-a-constant algorithms} \Rightarrow \text{Ex: binary search}$$

$$\text{if } a > 1 \Rightarrow \text{divide-and-conquer algorithms} \Rightarrow \text{merge and quick sort}$$

$$\textcircled{1} \Rightarrow x(n) = a^{n-1} \cdot b + \sum_{i=2}^n a^{n-i} \cdot f(i) \quad [\text{By backwards substitution}]$$

$$\textcircled{a} \text{ if } a = 1 \text{ and } f(n) = 1 \Rightarrow x(n) = b + \sum_{i=2}^n 1 = b + (n-1) \in \Theta(n)$$

$$\textcircled{a} \text{ if } a = 1 \text{ and } f(n) = \log n \Rightarrow x(n) = b + \sum_{i=2}^n \log i = b + \log(n!) \in \Theta(n \cdot \log n)$$

$$\textcircled{2} \Rightarrow \text{If } n \text{ is not a power of } b, \text{ we'll obtain something like } x(n) = a \cdot x\left(\left\lfloor \frac{n}{b} \right\rfloor\right) + \dots$$

$$\text{Assume that } n = b^k \Rightarrow x(n) = a^k \cdot c + \sum_{i=0}^{k-1} a^i \cdot f\left(\frac{n}{b^i}\right)$$

⊗ if $f(n) = d$

$$\Downarrow$$

$$x(n) = a^k \cdot c + d \cdot \sum_{i=0}^{k-1} a^i$$

$$a^k = a^{\log_b n}$$

$$= (b^{\log_b a})^{\log_b n} \Rightarrow a^k = n^{\log_b a}$$

$$\begin{aligned} & \text{again, it depends on this function} \\ & x(n) = n^{\log_b a} \cdot \left(c + d \cdot \left(\frac{n^{\log_b a} - 1}{a - 1} \right) \right) \\ & = n^{\log_b a} \cdot \left(c + \frac{d}{a-1} \right) - \frac{d}{a-1} \in \Theta(n^{\log_b a}) \end{aligned}$$

08.11.2022

Tuesday
Lecture 12

$$\text{If } a = b \Rightarrow n^{\log_b a} \cdot c + d \cdot n \cdot \sum_{i=0}^{k-1} 1 = c \cdot n + d \cdot n \cdot k$$

$$= cn + dn \cdot \log_b n$$

$$\in \Theta(n \log n)$$

This is the
case for
merge sort⊗ We have obtained these results only for $n = b^k$ ⊗ However, since all of these functions are Θ -invariant and non-decreasing, we can generalize them for all n .

09.11.2022

Wednesday

Lecture 13

Master Theorem: Let $x(n)$ be an eventually non-decreasing function that satisfies the recurrence relation:

$$x(n) = a \cdot x\left(\frac{n}{b}\right) + f(n); \text{ where } n = b^k \ (k=1, 2, \dots),$$

$$\text{and } x(1) = c, \text{ where } a > 1, b > 2, c > 0$$

If $f(n) \in \Theta(n^d)$, where $d \geq 0$, then

$$x(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \forall n \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Merge
Sort

$$x(n) = 2 \cdot x\left(\frac{n}{2}\right) + \underbrace{f(n)}_n$$

$$a=2$$

$$b=2$$

$$f(n) \in \Theta(n)$$

$$d=1$$

$$b^d = 2^1 = 2 = a$$

Quick Sort Invented by Hoare in 1960s

procedure Quicksort ($L[\text{low}:\text{high}]$)

if $\text{high} > \text{low}$ then

call Rearrange ($L[\text{low}:\text{high}], \text{position}$)

call Quicksort ($L[\text{low}:\text{position}-1]$)

call Quicksort ($L[\text{position}+1:\text{high}]$)

endif

if $\text{right} < \text{left}$ then
call interchange

($L[\text{left}], L[\text{right}]$)

end while

position = left

$L[\text{low}] = L[\text{position}]$

$L[\text{position}] = x$

end

procedure Rearrange ($L[\text{low}:\text{high}], \text{position}$)

right = low

left = high + 1

$x = L[\text{low}]$

while $\text{right} < \text{left}$ do:

repeat $\text{right} = \text{right} + 1$

until $L[\text{right}] \geq x$

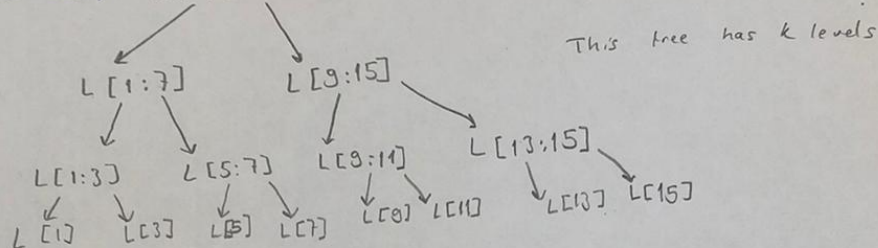
repeat $\text{left} = \text{left} - 1$

until $L[\text{left}] \leq x$

Basic operation: $\left. \begin{array}{l} L[\text{right}] > x \\ L[\text{left}] \leq x \end{array} \right\} \begin{array}{l} \text{comparisons} \\ \text{between} \\ \text{list elements} \end{array}$

⊗ Rearrange module always performs high-low+2 operations.

Best case; $L[1:15]$



if the list is divided into equal parts each time, this is the best case.

In Rearrange, always high-low+2 comparisons will be performed.

The tree has k levels

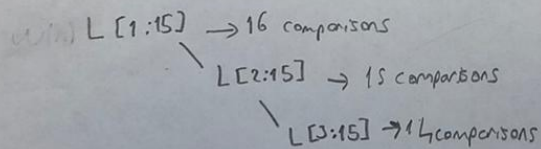
In $k-1$ of these levels, there are $n+1$ comparisons,

$$B(n) = (k-1)(n+1) = (\log_2(n+1)-1)(n+1) \in \Theta(n \log n) \quad \text{for } n=2^k-1$$

The functions are non-decreasing and Ω -invariant. Hence, we can generalize this result for all n .

Worst case

For quicksort, if the list is already sorted, this is the worst case



$$W(n) = \sum_{i=3}^{n+1} i \in \Theta(n^2)$$

$$L[14:15] \rightarrow 3 \text{ comparisons}$$